Módulo 0: Fundamentos e Ambiente de Desenvolvimento

Neste módulo, vamos aprender a preparar o ambiemte de desenvolvimento e conhecer as ferramentas e tecnologias básicas utilizadas na criação de páginas web. O conteúdo será apresentado de forma gradual, com exemplos simples para que cada conceito possa ser praticado.

1. Como funciona um código?

Um programa é um conjunto de instruções binárias que o computador executa para realizar tarefas. As Linguagens de programação são responsáveis por fazerem a tradução da nossa escrita semântica para o binário do computador.
Para escrever o código, utilizaremos o Visual Studio Code(VS Code), um editor que oferece recursos para facilitar a organização dos arquivos.

Instalando o VS Code

1. Acesse o site oficial: https://code.visualstudio.com/
2. Baixe a versão referente ao seu sistema operacional (Windows, macOS ou Linux).
3. Execute o instalador e siga as instruções apresentadas.
4. Abra o VS Code e crie uma pasta chamada meu-primeiro-projeto-web.

2. GIT E GITHUB

O que é Git?

Git é uma ferramenta de controle de versão. Ela registra alterações feitas em um projeto, permitindo acompanhar sua evolução e consultar versões anteriores.
Imagine que você esteja desenvolvendo uma calculadora. Primeiro, ela realiza somas; depois, você adiciona subtração e multiplicação. Se uma nova alteração causar um erro, o histórico do Git ajuda a identificar a mudança e recuperar uma versão anterior do projeto.

O que é GitHub?

GitHub é uma plataforma online que permite hospedar repositórios Git, compartilhar projetos e colaborar com outras pessoas.
Resumindo:

- Git registra o histórico de alterações.
- GitHub permite hospedar repositórios online e colaborar com outros desenvolvedores.

Instalando e configurando o Git

1. Acesse https://git-scm.com/ e baixe o instalador correspondente ao seu sistema.

2. Execute o instalador. Para começar, as opções padrão geralmente são suficientes.
3. Abra o terminal do computador ou o terminal integrado do VS Code.
4. Digite o comando abaixo para verificar a instalação:

```
git --version
```

Para configurar o nome e o e-mail utilizados nos commits, execute os comandos a seguir, substituindo os exemplos pelos seus dados:

```
git config --global user.name "Seu Nome"
git config --global user.email "seuemail@exemplo.com"
```

3. As Tecnologias básicas da programação web

As três tecnologias que vamos conhecer neste módulo são:

- HTML: estrutura e organiza o conteúdo da página.
- CSS: define a aparência, como cores, tamanhos e espaçamentos.
- JavaScript: adiciona comportamentos, como responder a cliques e alterar textos.

Podemos usar uma analogia para entender essa divisão: o HTML funciona como a estrutura de uma casa, o CSS define sua aparência e o JavaScript permite criar interações. Na prática, as três tecnologias trabalham em conjunto.

4. HTML

HTML significa HyperText Markup Language ou seja ela é uma linguagem de marcação. Ele é utilizado para organizar o conteúdo de uma página por meio de elementos, geralmente escritos com tags.
Exemplo básico de um arquivo HTML:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
   <meta charset="UTF-8">
   <title>Meu primeiro site</title>
</head>
<body>
   <div>
      <h1>Bem-vindo ao meu site!</h1>
      <p>Este é o meu primeiro parágrafo criado no curso.</p>
      <a href="https://google.com">Clique aqui para ir ao Google</a>
   </div>
</body>
</html>
```

Entendendo a estrutura:

- <!DOCTYPE html> informa ao navegador que o documento utiliza HTML moderno.
- <html> envolve todo o conteúdo da página. O atributo lang="pt-BR" informa que o idioma principal é português do Brasil.
- <head> contém os metadados sobre a página, como seu título e configurações.

- <meta charset="UTF-8"> define a codificação de caracteres, ajudando a exibir corretamente acentos e símbolos.
- <title> define o título que aparece na aba do navegador.
- <body> contém o conteúdo apresentado na página.

4.1. Tags essenciais

Para este início, não precisamos decorar todas as tags HTML. Vamos começar pelas mais básicas que iremos utilizar neste curso.

<h1> e <h2>

A tag <h1> representa o título principal da página ou do conteúdo. A tag <h2> representa um título de seção ou subtítulo. Existem níveis de <h1> até <h6>, que devem ser usados de acordo com a hierarquia do conteúdo.
Exemplo:

```
<h1>Meu site</h1>
<h2>Sobre mim</h2>
```

<p>

A tag <p> é utilizada para representar um parágrafo de texto.
Exemplo:

```
<p>Estou aprendendo desenvolvimento web.</p>
```

<a>

A tag <a> cria um link para outra página ou endereço. O atributo href define o destino.
Exemplo:

```
<a href="https://google.com">Acessar o Google</a>
```


A tag agrupa elementos, funcionando como um contêiner genérico. Ela é útil quando queremos organizar elementos ou aplicar estilos a um grupo, mas não descreve por si só o significado do conteúdo.
Exemplo:

```
<div>
  <h2>Sobre o curso</h2>
  <p>Este conteúdo está agrupado em uma div.</p>
</div>
```


A tag <img> exibe uma imagem. O atributo src indica o caminho da imagem e alt descreve seu conteúdo.
Exemplo:

```
<img src="imagem.jpg" alt="Descrição da imagem">
```

 e

A tag <ul> cria uma lista com marcadores, e cada <li> representa um item da lista.
Exemplo:

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

<button>

A tag <button> cria um botão. Ele pode ser utilizado, por exemplo, para iniciar uma ação por meio de JavaScript.
Exemplo:

```
<button>Clique aqui</button>
```

<link>

A tag <link> permite relacionar o HTML a um arquivo externo, como uma folha de estilos CSS. Ela costuma ficar dentro do <head>.
Exemplo:

```
<link rel="stylesheet" href="style.css">
```

4.2. Atributos básicos

Atributos fornecem informações adicionais aos elementos HTML. Alguns que usaremos com frequência são:

- href: define o destino de um link.
- src: define o caminho de uma imagem ou recurso.
- alt: fornece uma descrição textual para uma imagem.
- class: atribui uma classe que pode ser utilizada para aplicar estilos CSS.
- id: identifica um elemento específico, podendo ser utilizado para encontrá-lo com JavaScript.

Exemplo:

```
<h1 class="titulo" id="mensagem">Olá, mundo!</h1>
```

Nesse exemplo, o elemento possui a classe titulo e o identificador mensagem. A classe poderá ser utilizada no CSS, enquanto o ID poderá ser utilizado, por exemplo, para localizar esse título com JavaScript.

4.3. HTML semântico

HTML semântico consiste em escolher elementos de acordo com o significado do conteúdo, e não apenas pela aparência ou pela necessidade de agrupá-lo. Isso torna o código mais organizado e facilita sua compreensão por desenvolvedores, navegadores e tecnologias assistivas.
Por exemplo, poderíamos montar toda a página usando apenas . Entretanto, quando existem elementos mais específicos, vale utilizá-los para deixar clara a função de cada parte.
As principais tags semânticas que vamos conhecer são:

- <header>: representa o cabeçalho da página ou de uma seção. Pode conter o título e outras informações introdutórias.
- <nav>: agrupa links de navegação, como os links para as principais áreas do site.
- <main>: identifica o conteúdo principal da página.
- <section>: agrupa conteúdos relacionados, geralmente organizados em torno de um assunto.
- <article>: representa um conteúdo que pode ser compreendido de forma independente, como uma notícia ou publicação.
- <footer>: representa o rodapé da página ou de uma seção.

Exemplo de estrutura semântica:

```
<body>
  <header>
    <h1>Meu site</h1>
  </header>

  <nav>
    <a href="#sobre">Sobre</a>
    <a href="#contato">Contato</a>
  </nav>

  <main>
    <section id="sobre">
      <h2>Sobre o site</h2>
      <p>Este é o conteúdo principal da página.</p>
    </section>

    <section id="contato">
      <h2>Contato</h2>
      <p>Entre em contato conosco.</p>
    </section>
  </main>
```

```
    <footer>
        <p>Meu site - 2026</p>
    </footer>
</body>
```

Nesse exemplo, cada elemento indica a finalidade de uma parte da página: o cabeçalho fica em <header>, os links de navegação em <nav>, o conteúdo principal em <main> e as seções relacionadas em <section>. O rodapé fica em <footer>.
Diferente das tags semânticas a é um contêiner genérico e não informa qual é a função do conteúdo. Já tags como <header>, <nav>, <main> e <footer> descrevem a função da região. Isso não significa que a seja errada: ela continua sendo útil quando não existe uma tag semântica mais adequada.

5. CSS

CSS significa Cascading Style Sheets (Folhas de Estilo em Cascata). Ele define a aparência dos elementos HTML, como cores, tamanhos, fontes e espaçamentos.
Exemplo:

```
h1 {
    color: blue;
}
```

Nesse exemplo, h1 é o seletor e color: blue; é a declaração que define a cor azul para o título.

5.1. Formas de utilizar CSS

CSS inline

Escrito diretamente no elemento HTML por meio do atributo style.
Exemplo:

```
<h1 style="color: red;">Meu título</h1>
```

CSS interno

Escrito no próprio arquivo HTML, dentro da tag <style>, geralmente no <head>.
Exemplo:

```
<style>
    h1 {
        color: red;
    }
</style>
```

CSS externo

Escrito em um arquivo separado, normalmente com a extensão .css. Essa será a forma

preferencial nos exemplos do curso, pois mantém o HTML e o CSS organizados.
Exemplo:
Arquivo style.css:

```
h1 {
    color: blue;
}
```

No arquivo index.html, conectamos o CSS dentro do <head>:

```
<link rel="stylesheet" href="style.css">
```

5.2. Classes no CSS

Classes permitem aplicar um estilo a elementos escolhidos. No HTML, utilizamos o atributo class; no CSS, escrevemos o nome da classe precedido por ponto (.).
Exemplo:
Arquivo style.css:

```
.titulo {
    color: blue;
}
```

```
<h1 class="titulo">Meu título azul</h1>
<h1>Outro título</h1>
```

A regra .titulo será aplicada somente ao elemento que possui class="titulo". Assim, podemos escolher quais elementos receberão determinado estilo.
Algumas propriedades CSS importantes para este módulo:

- color: cor do texto.
- background-color: cor de fundo.
- font-size: tamanho da fonte.
- width e height: largura e altura.
- margin: espaçamento externo.
- padding: espaçamento interno.
- display: define como o elemento será disposto no layout.
- gap: define o espaço entre itens em layouts como Flexbox.

6. FLEXBOX

Flexbox é um recurso do CSS que ajuda a organizar elementos dentro de um contêiner. Com ele, podemos posicionar itens em linha ou coluna e controlar o alinhamento e o espaço entre eles.
Exemplo de CSS:

```
.pai {
    display: flex;
    gap: 20px;
}
```

```
.filho {
    background-color: blue;
    width: 100px;
    height: 100px;
}
```

Exemplo de HTML:

```
<div class="pai">
    <div class="filho"></div>
    <div class="filho"></div>
    <div class="filho"></div>
</div>
```

A classe pai recebe display: flex, ativando o Flexbox para organizar seus elementos filhos. Por padrão, eles ficam em uma linha. A propriedade gap: 20px cria um espaço entre eles. A classe filho define a cor e as dimensões dos quadrados.

6.1. Alterando a direção com flex-direction

A propriedade flex-direction define a direção dos itens.
Exemplo:

```
.pai {
    display: flex;
    gap: 20px;
    flex-direction: column;
}
```

Os valores mais utilizados neste primeiro contato são:

- row: organiza os itens em linha. É o valor padrão.
- column: organiza os itens em coluna, um abaixo do outro.

6.2. Alinhamento básico

Também podemos utilizar:

- justify-content: controla a distribuição dos itens no eixo principal.
- align-items: controla o alinhamento dos itens no eixo transversal.
- gap: define o espaço entre os itens.

Exemplo:

```
.pai {
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 20px;
}
```

Experimente alterar os valores e observar como os elementos se movimentam. O resultado

depende da direção do Flexbox e do tamanho do contêiner.

7. JavaScript

JavaScript é uma linguagem de programação utilizada para adicionar comportamentos à página. Com ela, podemos responder a cliques, alterar textos e executar ações.

7.1. Primeiro exemplo: alerta

```
<button onclick="alert('Olá! Seja bem-vindo ao curso!')">
  Clique em mim!
</button>
```

A tag <button> cria o botão. O atributo onclick indica uma ação a executar quando ocorre um clique, e alert() exibe uma mensagem no navegador. Esse formato é útil para o primeiro exemplo; mais adiante, podemos separar o JavaScript do HTML.

7.2. Alterando um texto

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Exemplo com JavaScript</title>
</head>
<body>
  <h1 id="mensagem">Texto antigo</h1>

  <button onclick="mudarTexto()">Trocar mensagem</button>

  <script>
    function mudarTexto() {
      document.getElementById("mensagem").innerText = "Texto trocado!";
    }
  </script>
</body>
</html>
```

Entendendo o exemplo:

- function mudarTexto() cria uma função chamada mudarTexto.
- document.getElementById("mensagem") localiza o elemento que possui o ID mensagem.
- .innerText permite alterar o texto desse elemento.
- Ao clicar no botão, onclick chama a função e o título passa a exibir “Texto trocado!”.